

saddammont / 2026C2-G03-PROGRAMACI-N

<> Code

🕒 Issues

🔗 Pull requests

🤖 Agents

🕒 Actions

📁 Projects

📖 Wiki

🛡 Security and quality

📈 Insights

⚙ Settings

2026C2-G03-PROGRAMACI-N / Tarea / 2. Segundo análisis de casos – Fundamentos de Python.ipynb

⌵

⋮

saddammont

Create 2. Segundo análisis de casos – Fundamentos de Python.ipynb

d7c0352 · 8 minutes ago

🕒

310 lines (310 loc) · 9.55 KB

Tarea Semana 4: Segundo Análisis de Casos

Código Limpio, Datos de Texto y Validación

Objetivo

Resolver cuatro ejercicios fundamentales aplicando entrada de datos, limpieza de texto, conversión de tipos, condicionales y salidas claras con f-strings.

Cada ejercicio debe demostrar el razonamiento antes del código. No se trata solo de que el programa funcione: debe verse claramente qué datos entran, qué proceso se aplica y qué salida se espera.

Formato obligatorio para cada ejercicio

Complete las siguientes partes:

- 1. **Problema:** describa qué debe resolver el programa.
- 2. **Entrada:** indique qué datos solicita al usuario.
- 3. **Proceso:** explique la fórmula, condición o validación usada.
- 4. **Salida esperada:** describa qué mensaje debe mostrar.
- 5. **Código:** escriba y ejecute la solución.

PreviewCodeBlame

Edit file...

In placeE

Open with...

github.dev.

GitHub Desktop

Entrega

- Entregue este notebook .ipynb completo.
- Exporte también el trabajo a PDF.
- Como práctica opcional, entregue uno de los ejercicios adicionalmente como archivo .py .

Ejercicios obligatorios

#	Ejercicio	Descripción	Habilidades clave
1	Calculadora de propina inteligente	Solicita total de cuenta y porcentaje de propina. Calcula propina y total final.	float , operaciones, f-strings
2	Detector de número par/impar	Solicita un entero e indica si es par o impar.	int , %, if/else
3	Clasificador de edad	Solicita edad y clasifica como menor, adulto o adulto mayor.	int , if/elif/else
4	Validador de compra con descuento	Solicita cliente, monto y si es cliente frecuente. Aplica descuento según reglas.	strings, float , and/or , f-strings

Use try-except en los ejercicios donde el usuario deba ingresar números.

Problema #1: Calculadora de propina inteligente

Problema

se necesita una calculadora para determinar el total de la propina

Entrada

Proceso

-

Salida esperada

-

Pruebas manuales

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso normal	total 1000 propina 15	15.0	15.0
Caso límite	total		
Caso inválido	total mil propina quince	error	error

```
In [ ]: # Escriba la solución aquí
total = float(input("El total de la cuenta es: $"))
propina = float(input("El porcentaje de la propina es (valores numericos positivos): "))
try:
    print("----- factura -----")
    print(f"subtotal = ${total}")
    print(f"Propina = {propina}")
    print(f"El total de la propina es {total - (total- (total*(propina/100)))}")
```

```
except ValueError:
    print("los valores deben de ser numeros positivos mayor que cero")
```

Problema #2: Detector de número par/impar

Problema

Solicita un entero e indica si es par o impar.

Entrada

-

Proceso

-

Salida esperada

-

Pruebas manuales

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso normal	24	numero par	numero par
Caso límite	0	numero par	numero par
Caso inválido	0.5	los numeros deben de ser enteros positivos	los numeros deben de ser enteros positivos

```
In [ ]: # Escriba la solución aquí

try:
    numero = int(input("Ingrese un numero entero positivo: "))
    valor = numero % 2
    if valor == 0 :
        print(" este numero es par")
    else:
        print(f"el valor no es un numero par. tu numero es {numero}")
except ValueError:
    print("los datos deben de ser enteros positivos")
```

Problema #3: Clasificador de edad

Problema

Solicita edad y clasifica como menor, adulto o adulto mayor.

Entrada

-

Proceso

-

Salida esperada

-

Pruebas manuales

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso normal	20	adulto	adulto
Caso limite	150	adulto mayor	adulto mayor
Caso inválido	-35	error	adulto

```
In [ ]: # Escriba la solución aquí
try:
    edad = int(input("Por favor ingresa tu edad en numero entero positivo: "))
    if 0 < edad <= 18:
        print(f"Tu edad es {edad} y estas clasificado como menor")
    elif edad < 60:
        print(f"Tu edad es {edad} y estas clasificado como adulto")
```

```
else:
    print(f"Tu edad es {edad} y estas clasificado como Adulto mayor")
except ValueError:
    print("los valores deben de ser enteros numericos positivos")
```

Problema #4: Validador de compra con descuento

Problema

Solicita cliente, monto y si es cliente frecuente. Aplica descuento según reglas.

Entrada

-

Proceso

-

Salida esperada

-

Pruebas manuales

Escenario	Datos de entrada	Resultado esperado	Resultado obtenido
Caso normal	si es cliente	descuento aplicado al monto	descuento aplicado
Caso limite	si el monto es cero	monto cero	monto cero
Caso inválido	monto negatico	un error	un monto negativo

```
In [ ]: # Escriba La solución aquí
try:
    cliente = input("ingrese su nombre si es cliente: ").capitalize().strip()
    monto = float(input("digite el monoton de la compra(numeros positivos): $"))
    cliente_frecuente = input("escriba SI si usted es cliente frecuente de caso contrario escriba NO").strip().lower()
    descuento = monto*(0.15)

    if cliente_frecuente == 'si':
        print(f"por ser un cliente leal {cliente} se agrego un descuento de {descuento} dando un total de {monto-descuento}")
    else:
        print(f"{cliente} su total seria {monto}")

except ValueError:
    print("debe ingresar el monto en numeros positivos, y si es cliente frecuente la palabra si o no")
```

Conclusión

Redacte una reflexión breve:

- ¿Qué validación le resultó más importante? r = la validacion de numeros y de valores flotante
- ¿Qué error encontró durante las pruebas? r = los valores negativos y las mala digitacion del usuario
- ¿Cómo mejoró la limpieza del código la calidad del programa? r = eneromemente ya que ayuda a tener mejor razonamiento cuando los valores on los esperados